AI Career Path Advisor

Complete Codebase Documentation

Emmerence Thesis Project

AUCA — Adventist University of Central Africa

Project Type:	Full-Stack Web Application
Backend:	Python / Django REST Framework
Frontend:	React / Vite / TypeScript
Database:	PostgreSQL
AI Engine:	Cosine Similarity Vectorization
Authentication:	JWT + OTP Email Verification
Repository:	https://github.com/andremugabo/career-advisor

June 16, 2026

Contents

Project Overview

The **AI Career Path Advisor** is an enterprise-grade, full-stack web application designed for the Emmerence Thesis Project at AUCA. The system connects student academic profiles and skill sets to industry career clusters using an AI-driven cosine similarity matching engine.

Problem Statement

Students at AUCA lack a systematic, data-driven tool to help them:

- Identify which career paths align with their academic skills and interests
- Understand skill gaps between their current competencies and career requirements
- Discover internship opportunities matched to their profile
- Receive personalized career guidance from advisors

Solution

The platform provides an integrated solution with three user portals:

1. **Student Portal** (12 pages): AI-powered career assessment, skill tracking, internship matching, and advisor messaging.
2. **Advisor Portal** (5 pages): Student monitoring, intervention tracking, and direct messaging to students.
3. **Admin Portal** (6 pages): User management, system analytics, RBAC permission editing, and platform settings.

System Architecture

The system follows a **decoupled client-server architecture**. The React frontend communicates with the Django backend exclusively through RESTful API endpoints, authenticated via JWT tokens.

Key architectural decisions:

- **Monorepo structure**: Both `backend/` and `frontend/` live in a single repository for simplified deployment.
- **UUID primary keys**: Every model uses `UUIDField` as the primary key (via `AbstractBaseEntity`) to prevent enumeration attacks and support distributed systems.
- **Audit trail**: Every model extends `AbstractAuditEntity`, automatically tracking `created_at`, `updated_at`, `created_by`, `updated_by`, `created_from_ip`, and `modified_from_ip`.
- **Role-Based Access Control (RBAC)**: Enforced at both the frontend (layout guards) and backend (DRF permission classes) levels.

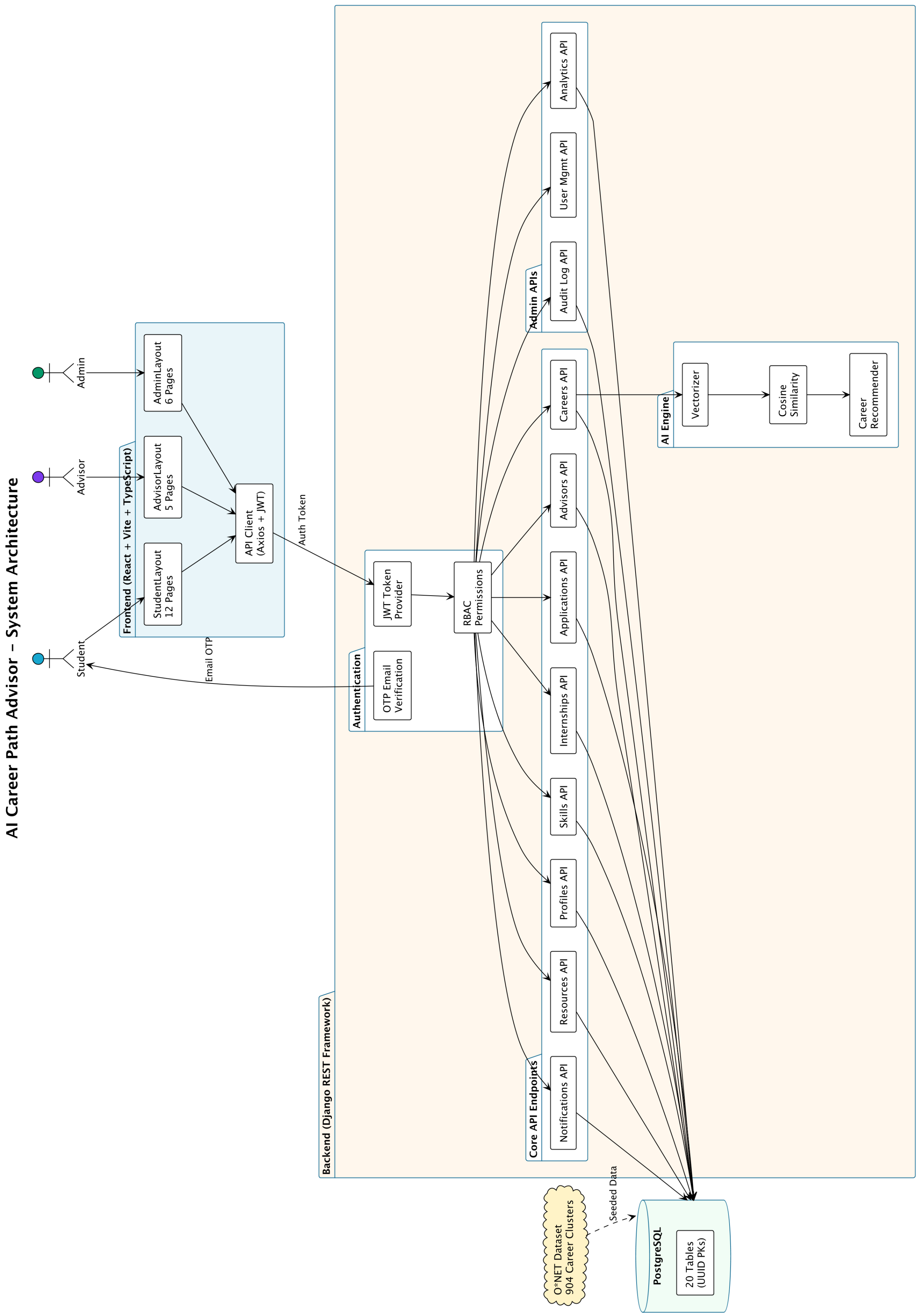


Figure 1: High-Level System Architecture

Technology Stack

Layer	Technologies
Backend Framework	Python 3.13, Django 6.x, Django REST Framework (DRF)
Database	PostgreSQL with UUID primary keys
Authentication	JWT via `rest_framework_simplejwt` with automatic token rotation
Email	Gmail SMTP for OTP verification and welcome emails
AI/ML	Custom cosine similarity engine (NumPy-free, pure Python)
API Documentation	OpenAPI 3.0 via `drf-spectacular`, Swagger UI
Frontend Framework	React 18+ with Vite build tool
Language	TypeScript (strict mode)
Styling	TailwindCSS utility classes
Routing	React Router v6 with RBAC layout guards
HTTP Client	Axios with JWT interceptor
Charts	Recharts for data visualization
Icons	Lucide React
Notifications	react-hot-toast

Backend Architecture

Directory Structure

```
backend/
  ai/                           # AI recommendation engine
    recommenders/
      career_recommender.py     # Main recommendation orchestrator
    similarity/
      vectorizer.py             # Skill-to-vector transformation
      cosine.py                 # Cosine similarity calculation
  apps/                         # 13 Django domain modules
    advisors/                   # Advisor model +
       StudentIntervention
    analytics/                  # SystemOverview, PermissionMatrix
       , Encryption
    applications/               # Student application tracking
    audit/                      # IP-based audit logging
    authentication/             # Registration, OTP email,
       password reset
    careers/                    # O*NET clusters, assessments,
       favorites
    internships/                # Job postings with AI match
       scoring
    notifications/              # AdvisorMessage (student-advisor
       messaging)
    profiles/                   # Student, AcademicPrograms models
```

```
    recommendations/            # AI match result storage and
       views
    resources/                  # Career resource library
    skills/                     # Skills dictionary, certs,
       StudentSkill
    users/                      # User model with role-based
       access
  config/                       # Django settings, URL routing,
     CORS, JWT
  core/                         # Abstract base models, pagination
     , exceptions
    models/
      base.py                   # AbstractBaseEntity (UUID PK)
      audit.py                  # AbstractAuditEntity (timestamps
         + IP)
  venv/                         # Python virtual environment
```

Base Model Hierarchy

Every model in the system inherits from a two-level abstract hierarchy:

Listing 1: AbstractBaseEntity — UUID Primary Key

```python
# core/models/base.py
import uuid
from django.db import models

class AbstractBaseEntity(models.Model):
    id = models.UUIDField(
        primary_key=True,
        default=uuid.uuid4,
        editable=False
    )
    class Meta:
        abstract = True
```

Listing 2: AbstractAuditEntity — Audit Fields

```python
# core/models/audit.py
class AbstractAuditEntity(AbstractBaseEntity):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    created_by = models.CharField(max_length=255, null=True)
    updated_by = models.CharField(max_length=255, null=True)
    created_from_ip = models.GenericIPAddressField(null=True)
    modified_from_ip = models.GenericIPAddressField(null=True)
    class Meta:
        abstract = True
```

This ensures every record in the database has:

- A globally unique, non-sequential UUID identifier

- Automatic creation and modification timestamps
- Traceability of which user and IP address created or modified the record

Database Schema (Entity Relationship Diagram)

The system contains **20 database tables** across 13 domain modules. All relationships use UUIDs as foreign keys. The full ERD is shown on the following landscape page.

Key Models Explained

Model	Module	Purpose
`User`	users	Core user with email, role (Student/Advisor/Admin), OTP verification
`Student`	profiles	Academic profile linked 1:1 to User. Contains GPA, bio, career goals, transcript
`AcademicPrograms`	profiles	University programs with M2M link to Skill (program_skills)
`Skill`	skills	Master dictionary of 533+ technical and soft skills
`StudentSkill`	skills	Junction table: student ↔ skill with proficiency level (1–5)
`Certification`	skills	Professional certifications (AWS, Google, etc.) linked to CareerClusters
`StudentCertification`	skills	Student's certification enrollment: Planning or Completed, with document upload
`CareerCluster`	careers	904 O*NET career occupations with skills, education/experience requirements
`CareerAssessment`	careers	Student's RIASEC assessment session
`FavoriteCareer`	careers	Bookmarked career clusters per student
`MatchResult`	recommendations	AI similarity scores + missing skill gaps per student-career pair
`Internship`	internships	Job postings linked to CareerClusters for AI matching
`Application`	applications	Student application to an internship with status pipeline
`Advisor`	advisors	Advisor profile linked 1:1 to User
`StudentIntervention`	advisors	Academic/career intervention logged by an advisor for a student
`AdvisorMessage`	notifications	Direct messages from advisor to student
`AuditLog`	audit	Change tracking: field name, old value, new value, IP address

Model	Module	Purpose
`Resource`	resources	Career development resources (guides, articles, templates)

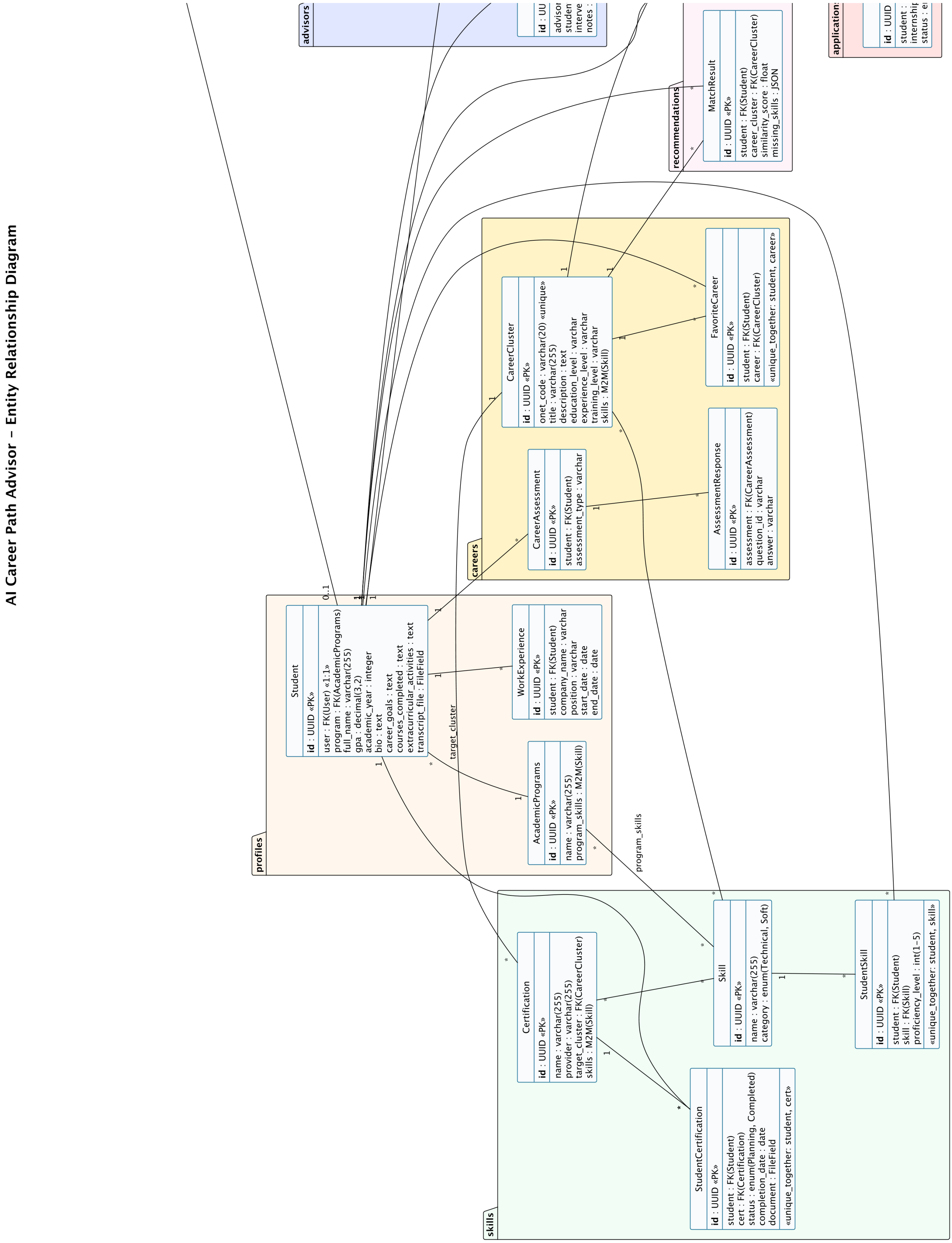


Figure 2: Complete Entity Relationship Diagram — 20 Tables Across 13 Domain Modules

AI Recommendation Engine

The AI engine is the core differentiator of this platform. It uses **cosine similarity** to calculate how closely a student's skill profile matches each of the 904 O*NET career clusters.

How It Works

Step-by-Step Process

1. **Skill Collection**: The system gathers the student's skills from three sources:
   - `StudentSkill` records (manually added by the student)
   - `AcademicPrograms.program_skills` (automatically from the student's enrolled program)
   - Assessment responses from the RIASEC questionnaire
2. **Vectorization**: The `Vectorizer` class creates a binary skill vector for the student. If the global skill dictionary has n skills, the student vector is an n-dimensional binary vector where $v_i = 1$ if the student has skill i, and $v_i = 0$ otherwise.
3. **Career Vectorization**: Each of the 904 `CareerCluster` records has its own M2M relationship with the `Skill` model. These are also vectorized into n-dimensional binary vectors.
4. **Cosine Similarity Calculation**: For each career cluster, the engine computes:

   $$similarity(S, C) = \frac{S \cdot C}{\|S\| \times \|C\|}$$

   Where S is the student vector and C is the career cluster vector. The result is a value between 0 (no match) and 1 (perfect match).
5. **Skill Gap Analysis**: For each recommended career, the engine identifies which skills the career requires that the student does not yet possess:

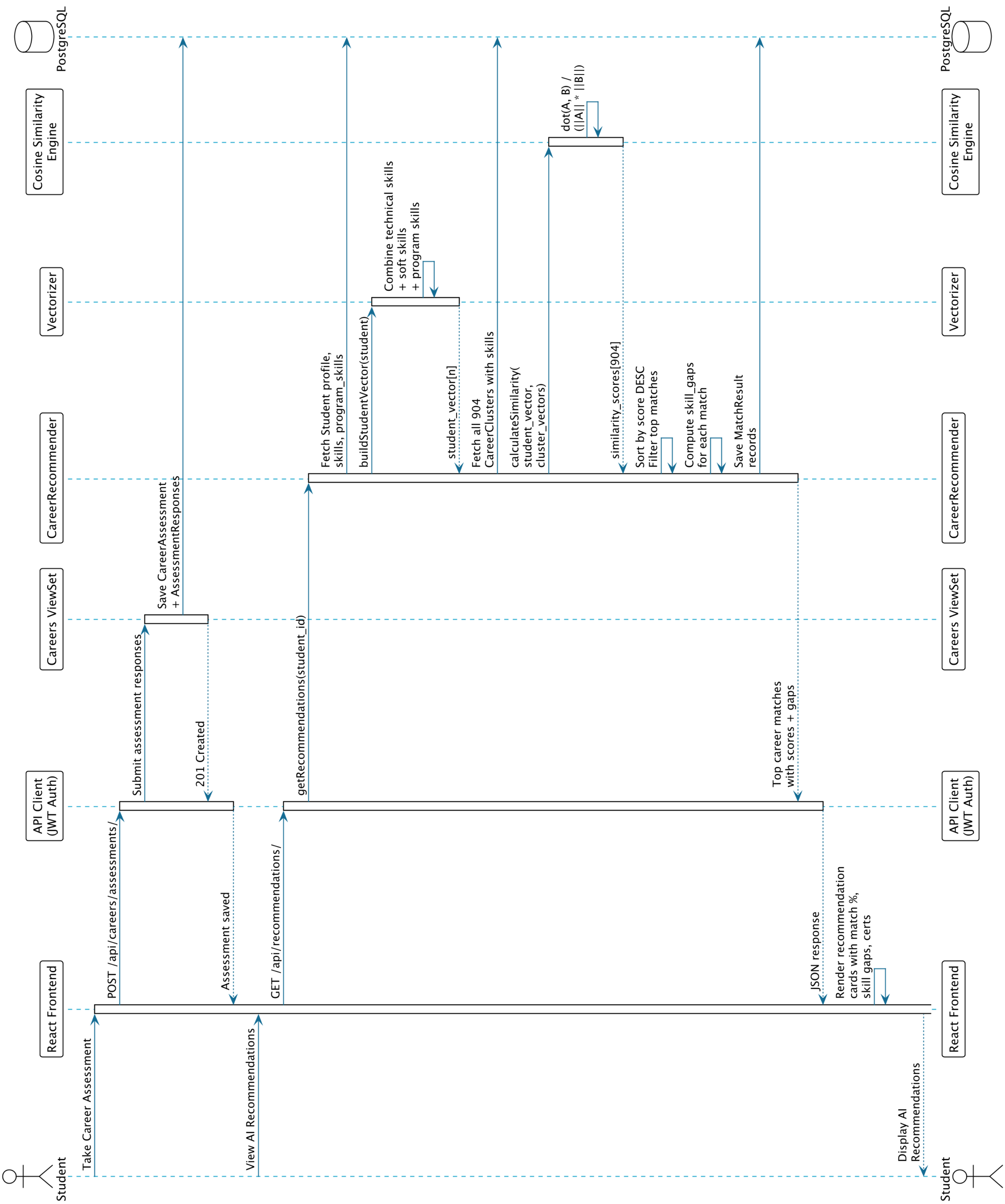
   $$missing\_skills = \{s \in C \mid s \notin S\}$$

6. **Ranking**: Results are sorted by similarity score in descending order. The top matches are saved as `MatchResult` records and returned to the frontend.

Key Implementation Files

File	Responsibility
`ai/similarity/vectorizer.py`	Builds binary skill vectors from student profiles and career clusters
`ai/similarity/cosine.py`	Computes cosine similarity between two vectors

`ai/recommenders/career_recommender.py`	Orchestrates the full pipeline: collect skills → vectorize → compute similarity → rank → return
`apps/recommendations/views.py`	DRF API endpoint that triggers the recommendation engine


Student
React Frontend
API Client (JWT Auth)
Careers ViewSet
CareerRecommender
Vectorizer
Cosine Similarity Engine
PostgreSQL
Take Career Assessment
POST /api/careers/assessments/
Submit assessment responses
Save CareerAssessment + AssessmentResponses
201 Created
Assessment saved
View AI Recommendations
GET /api/recommendations/
getRecommendations(student_id)
Fetch Student profile, skills, program_skills
buildStudentVector(student)
Combine technical skills + soft skills + program skills
student_vector[n]
Fetch all 904 CareerClusters with skills
calculateSimilarity(student_vector, cluster_vectors)
dot(A, B) / (||A|| * ||B||)
similarity_scores[904]
Sort by score DESC Filter top matches
Compute skill_gaps for each match
Save MatchResult records
Top career matches with scores + gaps
JSON response
Render recommendation cards with match %, skill gaps, certs
Display AI Recommendations

Authentication & Security

Authentication Flow

The system uses a multi-step authentication process combining JWT tokens with email OTP verification.

JWT Token Management

- **Token Obtain**: `POST /api/token/` returns an `access_token` (short-lived, 30 min) and a `refresh_token` (long-lived, 24 hours).
- **Token Refresh**: `POST /api/token/refresh/` silently refreshes the access token before expiry.
- **Frontend Storage**: Tokens are stored in `localStorage` and automatically attached to every API request via the Axios interceptor in `api/client.ts`.

Email OTP Verification

Upon registration, the system:

1. Generates a random 6-digit OTP code
2. Stores it in the `User.otp_code` field with a timestamp (`otp_created_at`)
3. Sends it via Gmail SMTP to the user's email
4. The OTP expires after 10 minutes
5. Once verified, `User.is_verified` is set to `True`

Role-Based Access Control (RBAC)

RBAC is enforced at two levels:

Backend Permission Classes

Listing 3: Custom DRF Permission Classes

```
class IsAdvisorOrAdmin(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.role in ['Advisor', 'Admin']

class IsSuperAdmin(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.role == 'Admin'
```

Frontend Layout Guards

Each portal is wrapped in a layout component that checks `localStorage.user_role`:

- `StudentLayout`: Redirects non-students to `/login`

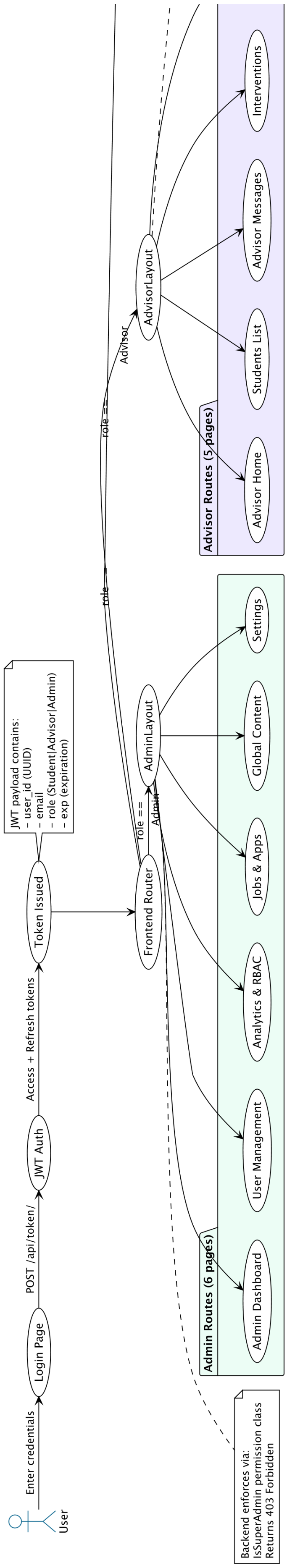


Figure 4: Role-Based Access Control Flow — Three-Tier Portal Routing

- `AdvisorLayout`: Redirects non-advisors to `/login` or `/dashboard`
- `AdminLayout`: Redirects non-admins to `/login`

Audit Logging

Critical profile changes (GPA, program, academic year) are automatically logged to the `AuditLog` table, recording:

- Which user made the change
- The field name, old value, and new value
- The client's IP address
- Timestamp of the change

Admins can export the full audit log as CSV via `GET /api/audit/export-csv/`.

Frontend Architecture

Component Architecture

The frontend follows a **feature-based architecture** where each page is a self-contained component that communicates with the backend through a dedicated service layer.

Directory Structure

```
frontend/src/
  api/
    client.ts              # Axios instance with JWT interceptor
  components/
    Button.tsx             # Reusable button (primary/secondary/
       danger)
    Card.tsx               # Glass-morphic card wrapper
    Pagination.tsx         # Smart windowed pagination
    Sidebar.tsx            # Role-aware navigation sidebar
    Topbar.tsx             # Header bar with user info
    ErrorBoundary.tsx      # Graceful error handling wrapper
  features/
    admin/                 # 6 admin pages
    advisors/              # 5 advisor pages
    applications/          # ApplicationsTracker
    auth/                  # Login, Register, VerifyEmail, etc.
    careers/               # Comparison, Visualization, Favorites
    dashboard/             # Student Dashboard
    misc/                  # StudentMessages, LandingPage, 404
    profile/               # StudentProfile
    resources/             # ResourceLibrary
    skills/                # SkillsCerts
    index.ts               # Barrel exports for all features
  layouts/
    StudentLayout.tsx      # RBAC guard for Student role
    AdvisorLayout.tsx      # RBAC guard for Advisor role
    AdminLayout.tsx        # RBAC guard for Admin role
    AuthLayout.tsx         # Public auth pages layout
  routes/
    AppRoutes.tsx          # All 30 routes with layout grouping
  services/
    student.service.ts     # Student API methods
    advisor.service.ts     # Advisor API methods
    admin.service.ts       # Admin API methods
    skill.service.ts       # Skills CRUD API methods
  types/
    index.ts               # TypeScript interfaces
```

API Client

The API client (`api/client.ts`) is an Axios instance configured to:

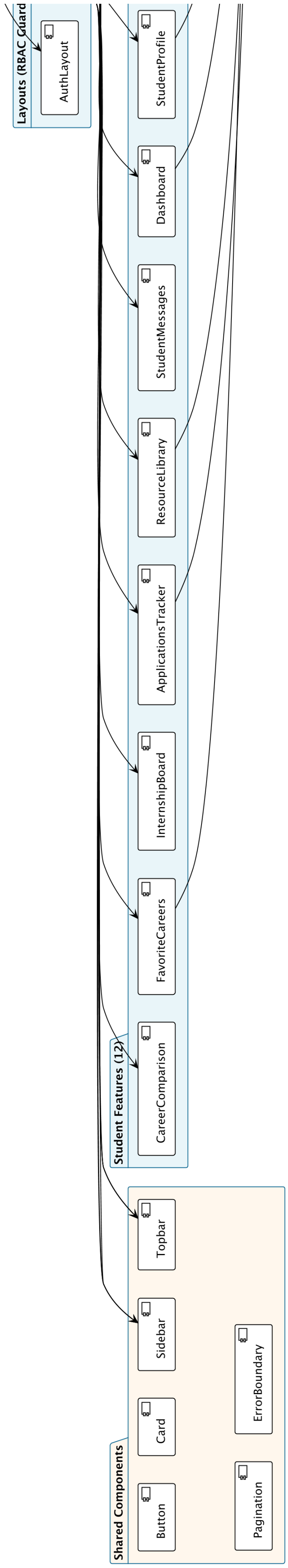


Figure 5: Frontend Component Architecture — 24 Pages, Service Layer, API Client

1. Automatically attach the JWT `Authorization: Bearer <token>` header to every request
2. Point all requests to `http://localhost:8000` (the Django backend)
3. Handle error responses with structured error messages

Service Layer Pattern

Each portal has a dedicated service file that encapsulates all API calls:

Listing 4: Service Method Examples

```
# student.service.ts
getProfile()            -> GET   /api/profiles/me/
updateProfile(data)     -> PATCH /api/profiles/me/
getRecommendations()    -> GET   /api/recommendations/
getFavoriteCareers()    -> GET   /api/careers/favorites/
getApplications(page)   -> GET   /api/applications/?page=N
getResources(page)      -> GET   /api/resources/?page=N

# advisor.service.ts
getStudents(page)       -> GET   /api/advisors/students/
createIntervention(d)   -> POST  /api/advisors/interventions/
sendMessage(data)       -> POST  /api/notifications/advisor-
   messages/

# admin.service.ts
getSystemOverview()     -> GET   /api/analytics/overview/
getPermissionMatrix()   -> GET   /api/analytics/permission-matrix/
getUsers(page)          -> GET   /api/admin/users/?page=N
exportAuditLogsCsv()    -> GET   /api/audit/export-csv/
```

API Endpoints Reference

Authentication Endpoints

Method	Endpoint	Description
POST	/api/token/	Obtain JWT access + refresh tokens
POST	/api/token/refresh/	Refresh access token
POST	/api/auth/register/	Register new user
POST	/api/auth/verify-email/	Verify email with OTP
POST	/api/auth/forgot-password/	Request password reset OTP
POST	/api/auth/reset-password/	Reset password with OTP

Student Endpoints

Method	Endpoint	Description
GET	/api/profiles/me/	Get current student profile
PATCH	/api/profiles/me/	Update profile fields
POST	/api/profiles/upload-transcript/	Upload PDF transcript
GET	/api/skills/	List all skills (searchable)
GET/POST	/api/skills/my/	Student's skills CRUD
GET	/api/certifications/my/	Student's certifications
GET	/api/recommendations/	AI career recommendations
POST	/api/careers/assessments/	Submit career assessment
GET/POST	/api/careers/favorites/	Toggle favorite careers
GET	/api/internships/	Browse internships
POST	/api/applications/	Submit application
GET	/api/applications/	List applications
GET	/api/resources/	Browse resources
GET	/api/notifications/student-messages/	View advisor messages

Advisor Endpoints

Method	Endpoint	Description
GET	/api/advisors/students/	List all student profiles
POST	/api/advisors/interventions/	Log an intervention
GET	/api/advisors/interventions/	Intervention history
POST	/api/notifications/advisor-messages/	Send message to student

Admin Endpoints

Method	Endpoint	Description
GET	/api/admin/users/	List all users
POST	/api/admin/users/	Create user
PATCH	/api/admin/users/<id>/	Update user
DELETE	/api/admin/users/<id>/	Delete user
GET	/api/analytics/overview/	System overview stats
GET	/api/analytics/permission-matrix/	RBAC permission matrix
GET	/api/analytics/encryption-status/	Security config status
GET	/api/audit/export-csv/	Export audit log as CSV
GET/POST	/api/skills/	Skills dictionary CRUD
GET/POST	/api/certifications/	Certifications CRUD
GET/POST	/api/internships/	Internship management
GET	/api/applications/	All student applications

Frontend Routes

Route Path	Portal	Component
`/`	Public	LandingPage
`/login`	Auth	Login
`/register`	Auth	Register
`/verify-email`	Auth	VerifyEmail
`/forgot-password`	Auth	ForgotPassword
`/reset-password`	Auth	ResetPassword
`/dashboard`	Student	Dashboard
`/profile`	Student	StudentProfile (editable)
`/skills`	Student	SkillsCerts
`/assessment`	Student	CareerAssessment
`/recommendations`	Student	RecommendationsPage
`/career-path`	Student	CareerVisualization
`/compare`	Student	CareerComparison
`/favorites`	Student	FavoriteCareers
`/internships`	Student	InternshipBoard
`/applications`	Student	ApplicationsTracker
`/resources`	Student	ResourceLibrary
`/messages`	Student	StudentMessages
`/advisor/home`	Advisor	AdvisorHome
`/students`	Advisor	AdvisorDashboard
`/advisor/messages`	Advisor	AdvisorNotifications
`/interventions`	Advisor	InterventionsHistory
`/advisor/resources`	Advisor	AdvisorResourceLibrary
`/admin/dashboard`	Admin	AdminDashboard
`/admin/users`	Admin	AdminUsers
`/admin/analytics`	Admin	AdminAnalytics
`/admin/internships`	Admin	AdminInternships
`/admin/content`	Admin	AdminContent
`/admin/settings`	Admin	AdminSettings

Data Seeding

The system uses Django management commands to seed initial data:

Command	What It Does
`python manage.py migrate`	Creates all 20 database tables from Django migrations
`python manage.py seed_onet`	Seeds 904 O*NET career clusters with associated skills from the dataset
`python manage.py seed_certs`	Seeds professional certifications (AWS, Google, etc.) mapped to career clusters
`python manage.py seed_internships`	Seeds mock internship postings linked to career clusters
`python create_admin.py`	Creates or resets the admin superuser account

Test Accounts

Role	Email	Password
Admin	`admin@auca.ac.rw`	`123`
Advisor	`test_advisor_api@auca.ac.rw`	`advisorpass123`
Student	`test_student_api@auca.ac.rw`	`studentpass123`

Critical Backend Configuration Files

This section documents every critical configuration file that controls how the backend connects to the database, authenticates users, handles errors, and logs audit trails.

Django Settings — `config/settings/base.py`

The settings system is split into a modular hierarchy inside `config/settings/`:

- `base.py` — Contains all shared configuration (database, JWT, CORS, installed apps, middleware, REST framework). This is the primary file.
- `dev.py` — Extends `base.py` with `DEBUG=True` and `ALLOWED_HOSTS=['*']`.
- `prod.py` — Placeholder for production overrides (HTTPS, strict CORS, etc.).
- `__init__.py` — Empty file enabling Python package imports.

When Django starts via `manage.py`, it uses `config.settings.dev` by default:

Listing 5: manage.py — Default Settings Module

```
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings.
    dev")
```

Database Connection

The database URL is loaded from the `.env` file using the `django-environ` library. This means the connection string is **never hardcoded** in the settings file:

Listing 6: Database Configuration in base.py

```
import environ

# Initialize environ and read .env file
env = environ.Env()
environ.Env.read_env(BASE_DIR / ".env")

# Parse the DATABASE_URL environment variable from .env
DATABASES = {
    "default": env.db("DATABASE_URL",
        default="sqlite:///db.sqlite3"
    )
}
```

The `DATABASE_URL` format is: `postgres://USER:PASSWORD@HOST:PORT/DBNAME`

Fallback behavior: If no `.env` file is present, it falls back to SQLite (`db.sqlite3`) for quick local testing.

JWT Token Configuration

JWT tokens are managed by `djangorestframework-simplejwt`. The settings control token lifetimes and rotation:

Listing 7: JWT Configuration in base.py

```
from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'AUTH_HEADER_TYPES': ('Bearer',),
    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',
}
```

Key details:

- Access tokens expire after **60 minutes**.
- Refresh tokens last **7 days**.
- `ROTATE_REFRESH_TOKENS=True`: Every refresh generates a new refresh token.
- `USER_ID_FIELD='id'`: Uses the UUID primary key as the token claim.

REST Framework Global Configuration

Listing 8: REST Framework Configuration in base.py

```
REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS':
        'drf_spectacular.openapi.AutoSchema',
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication
            .JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    ),
    'DEFAULT_PAGINATION_CLASS':
        'core.pagination.StandardResultsSetPagination',
    'EXCEPTION_HANDLER':
        'core.exceptions.custom_exception_handler',
}
```

This means:

- **All endpoints are protected by default** — only authenticated users (with valid JWT tokens) can access them. Public endpoints explicitly override this with `permissions.AllowAny`.
- All list endpoints are automatically paginated (20 items per page).
- Errors include the HTTP status code in the JSON response body.
- The Swagger documentation is auto-generated from code annotations.

Installed Apps and Middleware Stack

Listing 9: 13 Local Apps Registered in base.py

```
INSTALLED_APPS = [
    # Third Party
    "rest_framework",
    "corsheaders",
    "rest_framework_simplejwt",
    "drf_spectacular",
    # Django Built-ins (6 apps)
    "django.contrib.admin", ...
    # Local Apps (13 domain modules)
    "core",
    "apps.authentication",
    "apps.users",
    "apps.profiles",
    "apps.skills",
    "apps.careers",
    "apps.recommendations",
    "apps.internships",
    "apps.applications",
```

```
    "apps.advisors",
    "apps.analytics",
    "apps.resources",
    "apps.notifications",
    "apps.audit",
]
```

Listing 10: Middleware Stack (order matters)

```
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
    "core.middleware.GlobalAuditMiddleware",  # Custom
]
```

CORS: `CORS_ALLOW_ALL_ORIGINS = True` allows the React frontend on port 3000 to call the Django API on port 8000 without cross-origin blocking.

Email Configuration

Email settings (SMTP host, port, credentials) are loaded from the `.env` file:

Listing 11: Email Settings in base.py

```
EMAIL_BACKEND = env("EMAIL_BACKEND",
    default="django.core.mail.backends.console.EmailBackend")
EMAIL_HOST = env("EMAIL_HOST", default="localhost")
EMAIL_PORT = env.int("EMAIL_PORT", default=1025)
EMAIL_USE_TLS = env.bool("EMAIL_USE_TLS", default=False)
EMAIL_HOST_USER = env("EMAIL_HOST_USER", default="")
EMAIL_HOST_PASSWORD = env("EMAIL_HOST_PASSWORD", default="")
DEFAULT_FROM_EMAIL = env("DEFAULT_FROM_EMAIL",
    default="webmaster@localhost")
```

Fallback: Without a `.env` file, emails print to the console instead of sending via SMTP.

Environment File — `backend/.env`

The `.env` file contains **all secret credentials**. It is **not committed to Git**. Here is its structure:

Listing 12: Complete .env File Template

```
# Django Secret Key
SECRET_KEY="django-insecure-your-secret-key"

# PostgreSQL Connection String
```

```
DATABASE_URL="postgres://postgres:123@localhost:5432
    /career-advisor_db"

# Secure SMTP Configuration for Gmail
EMAIL_BACKEND="django.core.mail.backends.smtp
    .EmailBackend"
EMAIL_HOST="smtp.gmail.com"
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER="your-email@gmail.com"
EMAIL_HOST_PASSWORD="your-gmail-app-password"
DEFAULT_FROM_EMAIL="your-email@gmail.com"
```

How to get a Gmail App Password:

1. Go to Google Account → Security → 2-Step Verification (must be enabled)
2. Scroll to "App passwords" and generate a new one for "Mail"
3. Paste the 16-character password into `EMAIL_HOST_PASSWORD`

Custom User Model — `apps/users/models.py`

Django's built-in `User` model uses a username. This project replaces it with a **custom model using email as the login field**:

Listing 13: Custom User Model

```python
class User(AbstractBaseUser, PermissionsMixin,
           AbstractAuditEntity):
    ROLE_CHOICES = (
        ('Student', 'Student'),
        ('Advisor', 'Advisor'),
        ('Admin', 'Admin'),
    )
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=20,
        choices=ROLE_CHOICES, default='Student')
    mfa_enabled = models.BooleanField(default=False)
    is_verified = models.BooleanField(default=False)
    failed_login_attempts = models.IntegerField(default=0)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)

    objects = CustomUserManager()
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []
```

Registered in settings as: `AUTH_USER_MODEL = 'users.User'`

Global Audit Middleware — `core/middleware.py`

Every POST, PUT, PATCH, and DELETE request to any `/api/` endpoint is automatically logged by the `GlobalAuditMiddleware`:

Listing 14: GlobalAuditMiddleware — Automatic Request Logging

```
class GlobalAuditMiddleware(MiddlewareMixin):
    def process_response(self, request, response):
        if request.method in ['POST', 'PUT', 'PATCH', 'DELETE']:
            if request.path.startswith('/api/'):
                # Extract IP address (proxy-aware)
                x_forwarded_for = request.META.get(
                    'HTTP_X_FORWARDED_FOR')
                if x_forwarded_for:
                    ip = x_forwarded_for.split(',')[0].strip()
                else:
                    ip = request.META.get('REMOTE_ADDR')

                AuditLog.objects.create(
                    user=request.user if authenticated,
                    action=f"API_{request.method}",
                    details={
                        'path': request.path,
                        'method': request.method,
                        'status_code': response.status_code,
                        'ip_address': ip
                    },
                    created_from_ip=ip
                )
        return response
```

Key behavior:

- GET requests are **not logged** to avoid flooding the audit table.
- Auth-related paths (`/auth/login/`, `/auth/register/`) get sanitized labels.
- The client IP is extracted with proxy-awareness (`X-Forwarded-For` header).

Pagination — `core/pagination.py`

Listing 15: Standard Pagination Configuration

```
class StandardResultsSetPagination(PageNumberPagination):
    page_size = 20
    page_size_query_param = 'page_size'
    max_page_size = 100
```

Every list endpoint returns 20 items per page. The client can override this with `?page_size=50` (up to 100).

Exception Handler — `core/exceptions.py`

Listing 16: Custom Exception Handler

```python
def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)
    if response is not None:
        response.data['status_code'] = response.status_code
    return response
```

This adds a `status_code` field to every error JSON response, making it easier for the frontend to inspect errors programmatically.

Frontend API Client — `frontend/src/api/client.ts`

The frontend communicates with the backend through a centralized API client:

Listing 17: Frontend API Client (TypeScript)

```typescript
const BASE_URL = 'http://localhost:8000/api';

export async function apiFetch<T>(
    endpoint: string,
    options: RequestInit = {}
): Promise<T> {
    const token =
        localStorage.getItem('access_token');

    const headers = new Headers(options.headers);
    if (token)
        headers.set('Authorization',
            `Bearer ${token}`);
    if (!(options.body instanceof FormData))
        headers.set('Content-Type',
            'application/json');

    const response = await fetch(
        `${BASE_URL}${endpoint}`, { ...options, headers }
    );

    if (response.status === 401) {
        // Token expired: clear and redirect
        localStorage.removeItem('access_token');
        window.location.href = '/login';
    }
    // ... error handling
    return response.json();
}
```

Key design decisions:

- JWT token is read from `localStorage` and attached to every request as `Authorization: Bearer <token>`.
- 401 responses automatically clear the token and redirect to login.

- FormData requests (file uploads) skip the Content-Type header to let the browser set the boundary.

Admin Script — create_admin.py

This standalone script creates or resets the master admin account:

Listing 18: create_admin.py — Admin Account Setup

```python
os.environ.setdefault('DJANGO_SETTINGS_MODULE',
    'config.settings.base')
django.setup()
from apps.users.models import User

admin_email = 'admin@auca.ac.rw'
admin_pass = '123'

if not User.objects.filter(email=admin_email).exists():
    User.objects.create_superuser(
        admin_email, admin_pass)
else:
    user = User.objects.get(email=admin_email)
    user.set_password(admin_pass)
    user.save()
```

Key Python Dependencies

The project's requirements.txt contains the full Anaconda environment dump. The **critical project-specific packages** are:

Package	Version	Purpose
Django	5.2.1	Web framework
djangorestframework	latest	REST API endpoints
djangorestframework-simplejwt	5.x	JWT authentication
django-cors-headers	latest	Cross-origin requests (CORS)
django-environ	0.13.0	Environment variable parsing (.env)
django-otp	1.6.1	One-time password support
drf-spectacular	0.29.0	OpenAPI 3.0 / Swagger UI
psycopg2-binary	2.9.10	PostgreSQL database driver
PyJWT	latest	JWT token encoding/decoding
PyPDF2	3.0.1	PDF transcript parsing
reportlab	4.2.0	PDF report generation
fpdf	1.7.2	Lightweight PDF creation
pyotp	2.9.0	TOTP/HOTP OTP generation
sqlparse	latest	SQL query formatting

Frontend Dependencies — package.json

Package	Version	Purpose
`react`	18.3.1	UI component library
`react-dom`	18.3.1	DOM rendering
`react-router-dom`	6.23.1	Client-side routing with RBAC guards
`recharts`	2.12.7	Data visualization charts
`lucide-react`	0.379.0	SVG icon library
`react-hot-toast`	2.6.0	Toast notification system
`vite`	5.2.11	Build tool and dev server
`typescript`	5.2.2	Type-safe JavaScript
`tailwindcss`	3.4.19	Utility-first CSS framework

How to Run the Project — Step by Step

Follow these steps **in exact order** to set up and run the entire project from scratch on a new machine.

Step 1: Install Prerequisites

Before anything else, make sure these tools are installed on your system:

Tool	Min Version	How to Verify
Python	3.10+	`python3 -version`
PostgreSQL	13+	`psql -version`
Node.js	18+	`node -version`
npm	9+	`npm -version`
Git	Any	`git -version`

Step 2: Clone the Repository

```
git clone https://github.com/andremugabo/career-advisor.git
cd career-advisor
```

The project structure after cloning:

```
career-advisor/
  backend/          # Django REST API
  frontend/         # React + Vite client
  explaination/     # This documentation
  docs/             # Thesis LaTeX assets
  README.md
```

Step 3: Create the PostgreSQL Database

Open a terminal and create the database:

```
# Connect to PostgreSQL as the postgres user
psql -U postgres

# Inside the psql shell, run:
CREATE DATABASE "career-advisor_db";

# Verify it was created:
\l

# Exit psql:
\q
```

Note: The database name must be exactly `career-advisor_db` to match the connection string in the `.env` file. If your PostgreSQL password is not `123`, update the `.env` file accordingly.

Step 4: Configure the Backend Environment

Navigate to the backend directory and activate the Python virtual environment:

```
cd backend

# Activate the virtual environment
source venv/bin/activate
# On Windows: venv\Scripts\activate

# Verify Python is from the venv:
which python
# Should show: .../career-advisor/backend/venv/bin/python
```

Important: On macOS/Linux, if you see `zsh: command not found: python`, use `python3` instead, or activate the virtual environment first.

Step 5: Create the Environment File

Create a file called `.env` inside `backend/`:

Listing 19: backend/.env — Required Environment Variables

```
# Secret key for Django
SECRET_KEY="django-insecure-your-secret-key-here"

# PostgreSQL connection
DATABASE_URL="postgres://postgres:123@localhost:5432
    /career-advisor_db"

# Gmail SMTP for email OTP
EMAIL_BACKEND="django.core.mail.backends.smtp
    .EmailBackend"
EMAIL_HOST="smtp.gmail.com"
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER="your-email@gmail.com"
EMAIL_HOST_PASSWORD="your-16-char-app-password"
DEFAULT_FROM_EMAIL="your-email@gmail.com"
```

Replace the placeholder values with your actual PostgreSQL password and Gmail App Password.

Step 6: Run Database Migrations

Migrations create all 20 database tables from the Django model definitions:

```
python manage.py migrate
```

Expected output: 40+ lines of "Applying apps.users.0001_initial. . . OK" messages.

Step 7: Seed the Database

The system requires seed data to function. Run these commands **in order**:

```
# 1. Seed 904 O*NET career clusters with skills
#    (takes ~30 seconds, reads from dataset/ CSV files)
python manage.py seed_onet

# 2. Seed professional certifications (AWS, Google, etc.)
python manage.py seed_certs

# 3. Seed mock internship postings
python manage.py seed_internships
```

Step 8: Create the Admin User

```
python create_admin.py
```

Expected output: `Master superuser created: admin@auca.ac.rw`

Step 9: Start the Backend Server

```
python manage.py runserver
```

Expected output:

```
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Verify: Open `http://127.0.0.1:8000/api/docs/` in a browser. You should see the Swagger UI.

Step 10: Set Up the Frontend

Open a **new terminal tab** (keep the backend server running in the first tab):

```
cd career-advisor/frontend

# Install Node.js dependencies
npm install

# Start the development server
npm run dev
```

Expected output:

```
  VITE v5.2.11  ready in 300 ms
  ->  Local:   http://localhost:3000/
```

Step 11: Verify the Full System

Open http://localhost:3000 in a browser. You should see the landing page.

Login with the test accounts:

Role	Email	Password
Admin	`admin@auca.ac.rw`	`123`
Advisor	`test_advisor_api@auca.ac.rw`	`advisorpass123`
Student	`test_student_api@auca.ac.rw`	`studentpass123`

Each role redirects to a different dashboard:

- **Student** → `/dashboard` (12 pages accessible)
- **Advisor** → `/advisor/home` (5 pages accessible)
- **Admin** → `/admin/dashboard` (6 pages accessible)

Common Troubleshooting

Problem	Solution
`zsh: command not found: python`	Use `python3` or activate the venv first: `source venv/bin/activate`
`FATAL: database does not exist`	Create it: `psql -U postgres -c 'CREATE DATABASE "career-advisor_db"'`
`FATAL: password authentication failed`	Check the PostgreSQL password in your `.env` matches your `psql` password
`Port 8000 already in use`	Kill the existing process: `kill -9 $(lsof -t -i:8000)`
`Port 3000 already in use`	Kill the existing process: `kill -9 $(lsof -t -i:3000)`
Swagger page shows no end-points	Verify all 13 local apps are in `INSTALLED_APPS`
Frontend shows CORS error	Verify `CORS_ALLOW_ALL_ORIGINS = True` in base.py
Email OTP not received	Check Gmail App Password and `EMAIL_USE_TLS=True`

Testing

Automated Integration Tests

Run these commands from the `backend/` directory with the virtual environment activated:

```
# Full feature integration test (requires running server)
python manage.py test_all_features

# Server connectivity check
python dataset/test_server_connectivity.py
```

The integration test suite covers:

- User registration and authentication (JWT token flow)
- Profile creation and updating (with audit log verification)
- Skill and certification CRUD operations
- AI recommendation generation and ranking
- Internship search and application submission
- Advisor access restrictions (403 Forbidden for student tokens)
- Audit log completeness and IP tracking

Manual Testing via Swagger UI

1. Open `http://127.0.0.1:8000/api/docs/`
2. Click **Authorize** and enter a JWT access token (obtained from `POST /api/token/`)
3. Test any endpoint interactively

Summary of Functional Requirements

FR	Requirement	Implementation
FR-01	User Authentication	JWT + OTP email verification + role-based registration
FR-02	Student Profile Management	Editable profile with GPA, bio, transcript upload
FR-03	Skills Tracking	533+ skill dictionary, proficiency levels (1–5), program skills
FR-04	AI Career Matching	Cosine similarity engine with 904 O*NET clusters
FR-05	Skill Gap Analysis	Missing skills computed per career match
FR-06	Student-Advisor Messaging	AdvisorMessage model with read/unread tracking
FR-07	Certification Pathways	Certifications mapped to career clusters with document upload

FR	Requirement	Implementation
FR-08	Internship Matching	AI-sorted internships by student compatibility score
FR-09	Advisor Monitoring	Student list, interventions (8 types), direct messaging
FR-10	RBAC Security	Three-tier role system enforced at frontend + backend
FR-11	Audit Logging	IP-tracked change logs with CSV export
FR-12	Admin Management	User CRUD, analytics, permission matrix editor, system settings